TASK:6

Solve a **Map Coloring problem** using constraint satisfaction approach by applying following constraints

Aim: To Solve a Map Coloring problem using constraint satisfaction approach using Graphonline and visualago online simulator

Algorithm:
Step 1: Confirm whether it is valid to color the current vertex worth the current color (by checking whether any of its adjacent vertices are colored with the same color)
Step 2: If yes then color it and otherwise try a different color
Step 3: check if all vertices are colored or not
Step 4: If not then move to the next adjacent uncolored vertex
Step 5: Here backtracking means to stop further recursive calls on adjacent vertices.

. What is Map Coloring?

Map coloring is a problem where different regions of a map must be assigned different colors.

The main constraint is:

Two adjacent regions cannot have the same color.

In graph representation:

- Vertices → represent regions of the map.
- Edges → represent adjacency between regions.
- Colors → represent the possible values assigned to each vertex.

For this program:

- Number of vertices = 4
- Number of available colors = 3
- Each vertex can have color 1, 2, or 3.

---

3. Graph Used in the Program

The program uses the following adjacency matrix:

0 1 1 1
1 0 1 0
1 1 0 1

1 0 1 0

Let the four vertices be:

V1, V2, V3, V4

The connections are:

- V1 is connected to V2
- V1 is connected to V3
- V1 is connected to V4
- V2 is connected to V3
- V3 is connected to V4

There is no direct connection between V2 and V4.

Therefore, V2 and V4 are allowed to have the same color.

---

4. Algorithm Explanation

Step 1: Check whether the color is safe

The algorithm first checks whether the current vertex can be assigned a particular color.

For example, if we want to give color 1 to V2, the algorithm checks all vertices adjacent to V2.

If none of them has color 1, then the color is safe.

Step 2: Assign the color

If the selected color does not violate the constraint, it is assigned to the current vertex.

For example:

V1 = Color 1

Then the algorithm moves to the next vertex.

Step 3: Check whether all vertices are colored

The algorithm checks whether all four vertices have been assigned a color.

If all vertices are colored successfully, the algorithm returns True.

```
if v == self.v:
    return True
```

Here:

- v = current vertex
- self.v = total number of vertices

When v == self.v, all vertices have been processed.

Step 4: Move to the next vertex

If the current vertex has been successfully colored, the recursive function moves to the next vertex.

For example:

V1 → V2 → V3 → V4

The algorithm continues until every vertex has a valid color.

Step 5: Backtracking

If none of the available colors can be assigned to a vertex without violating the constraint, the algorithm backtracks.

For example:

color[v] = 0

This removes the previously assigned color.

The algorithm then tries another color.

So, backtracking means:

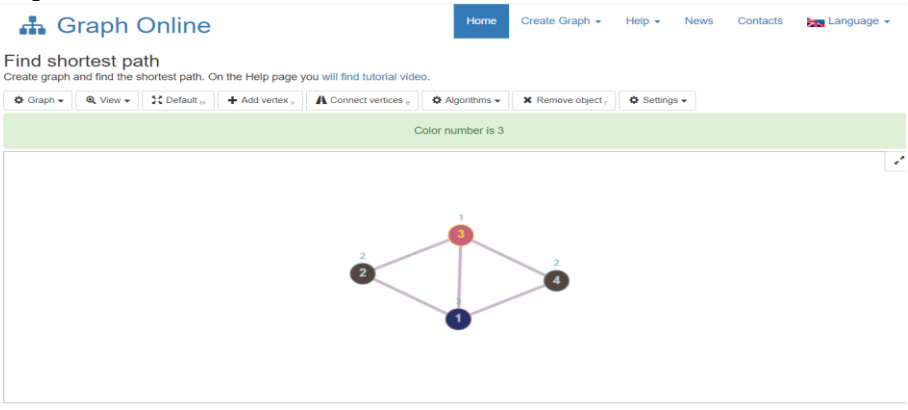
Undo the previous decision and try another possible decision.

Program:

```
class Graph:
    def __init__(self, vertices):
        self.v = vertices
        self.graph = [[0 for column in range(vertices)] for row in range(vertices)]
```

```
    # A utility function to check if the current color assignment is safe for vertex v
    def is_safe(self, v, color, c):
        for i in range(self.v):
            if self.graph[v][i] == 1 and color[i] == c:
                return False
        return True

    # A recursive utility function to solve m-coloring problem
    def graph_color_util(self, m, color, v):
        if v == self.v:
            return True

        for c in range(1, m+1):
            if self.is_safe(v, color, c):
                color[v] = c
                if self.graph_color_util(m, color, v+1):
                    return True
                color[v] = 0

    def graph_coloring(self, m):
        color = [0] * self.v
        if not self.graph_color_util(m, color, 0):
            return False

        # Print the solution
        print("Solution exists and following are the assigned colors:")
        for c in color:
            print(c, end=" ")

# Driver Code
if __name__ == '__main__':
    g = Graph(4)
    g.graph = [[0, 1, 1, 1], [1, 0, 1, 0], [1, 1, 0, 1], [1, 0, 1, 0]]
    m = 3
    # Function call
    g.graph_coloring(m)
```

Output:



Result:
Thus Solving a Map Coloring problem using constraint satisfaction approach using Graphonline and visulago online simulator was successfully executed and output was verified.

12. Why Backtracking is Required

Suppose the algorithm chooses a color for a vertex and later discovers that no color is available for another vertex.

Instead of stopping completely, it goes back to the previous vertex.

For example:

Choose Color
↓
Check Constraint
↓
Valid?
↓ Yes
Assign Color
↓
Next Vertex
↓
Conflict?
↓ Yes
Backtrack
↓
Remove Previous Color
↓
Try Another Color

Thus, backtracking helps the algorithm explore different combinations until a valid solution is found.